Python for ML

Instructor: Anshaj Khare

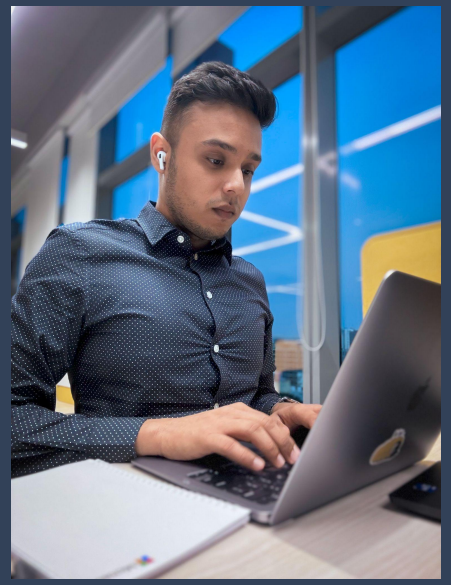
https://www.linkedin.com/in/anshaj-khare/

https://github.com/anshajk

Author: https://thecomputeblog.substack.com/

{ik} INTERVIEW KICKSTART

Introduction – Anshaj Khare



- ✔ I have been working on data science since 2017
- ✔ I've worked for 4 startups before joining Microsoft
- ✔ I have been working at Microsoft for more than 3 years.
- ✔ 8+ years in data science & engineering
- ✔ In my career, I've worked as a backend engineer, a devops engineer, a data engineer and as a Machine learning engineer
- ✔ Feel free to send a connection request on LinkedIn/Github if you'd like to connect - Anshaj Khare | LinkedIn, anshajk (Anshaj Khare) · GitHub

Welcome to today's class, before we begin, pop into the chat:

Hannah Chen

Data Scientist

Bay Area

Grammarly, ex-Uber

{ik} INTERVIEW KICKSTART

Structure of Class

1. **Pre-class video (Thursday)** - Introduction to NumPy
2. **Sunday Class - 4 hr (Sunday)**
3. **Post-class-video(s) -** Plotting
4. **Assignments -** MCQ & Coding Problems

Wednesday and Thursday Sessions

After the main class watch the **post class videos**

Wednesday - Technical Coaching Session

One hour of topical review on the class. This is an excellent time to reinforce Sunday's content or ask advanced questions, depending on your experience.

Thursday - Assignment Review Session

Two hours to review the assignment and the MCQs. Both assignments and MCQs are designed to reinforce the learning objectives which tie directly to knowledge and skills you will need in industry and in interviewing.

Optimize Your Experience

- ✅ Interact with your instructors via Sunday live class.
- ✅ Don’t be shy to speak up and get clarifications !

- ✅ Solve your assignments, MCQs and other assessments and get feedback (Thursday/Saturday review session)
- ✅ Use your resources! It’s your experience – what you put in is what you’ll get out.

Are you using all these IK support features?

- **Uplevel** - contains all the learning resources - videos, MCQs, assignments
- Optional **Post class videos** for additional reference
- **Technical coaching sessions** on Wednesday to clarify all your queries
- **Discord** community for peer learning
- **TAs** to help answer the queries in the live class
- **Support tickets** - Raise them any time you want any clarifications/ tech concerns , etc on UpLevel

Success Hacks

- Be prepared for the class by watching the pre class videos
- Attend every class; Sunday and Thursday
- Participate in class
- Attempt all the assignments,
- Actively advocate for yourself
- Keep your goal in mind
- Consistency is the key

Be patient with yourself !

Today's Agenda

1. NumPy Revisit

Break

2. Pandas Fundamentals

Break

3. Pandas Data Analysis

Break

4. Plotting with Matplotlib

{ik} INTERVIEW KICKSTART

Numpy Revisit

NumPy Revisit - What is NumPy (Numerical Python)

- NumPy is a Python library that provides support for *large, multi-dimensional* arrays and matrices.

- NumPy offers a large collection of high-level *mathematical* functions to operate on these arrays.

$1	$2	$5	$6	$7	$8	$2

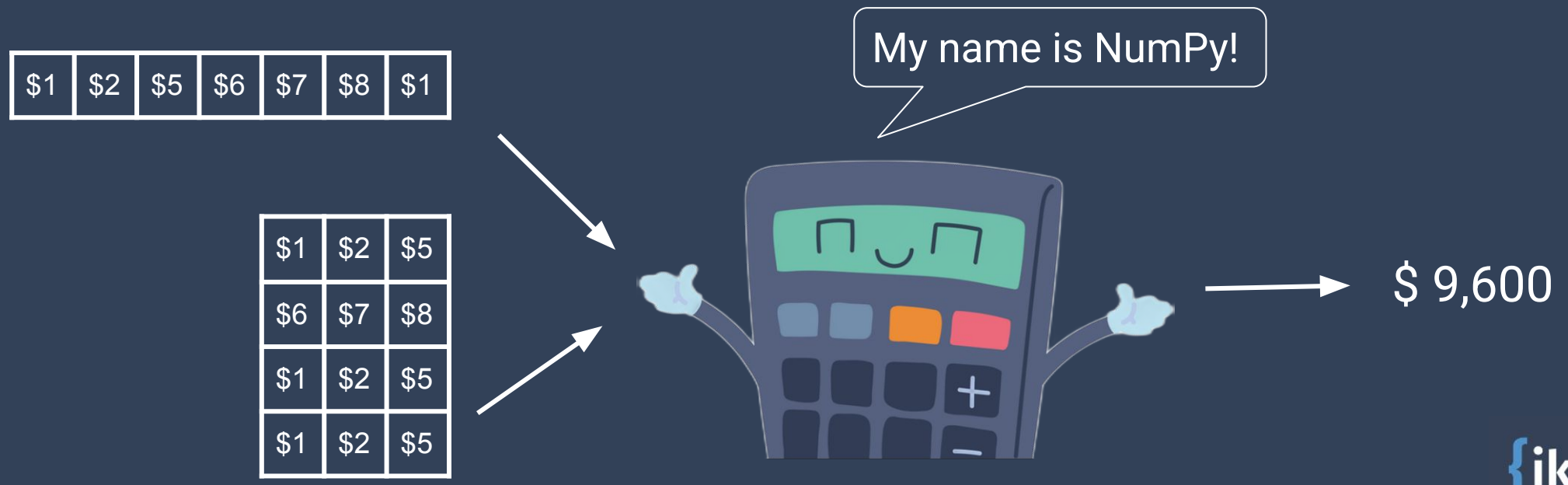
$1	$2	$5
$6	$7	$8
$1	$2	$5
$1	$2	$5

{ik} INTERVIEW KICKSTART

NumPy Revisit - What is NumPy (Numerical Python)

- NumPy is a Python library that provides support for *large, multi-dimensional* arrays and matrices.

- NumPy offers a large collection of high-level *mathematical* functions to operate on these arrays.



{ik} INTERVIEW KICKSTART

NumPy Revisit - Why Use Numpy (Numerical Python) ?

- **Speed**: NumPy is faster than regular Python lists.
- **Functionality**: It provides a lot of built-in functions that make mathematical calculations easy.
- **Efficiency**: With NumPy, you can perform operations on entire lists of data without having to use loops.

{ik} INTERVIEW KICKSTART

NumPy Revisit - Why Use Numpy (Numerical Python) ?

- **Speed**: NumPy is faster than regular Python lists.

Vs.

{ik} INTERVIEW KICKSTART

NumPy Revisit - Why Use Numpy (Numerical Python) ?

- **Speed**: NumPy is faster than regular Python lists.
- **Functionality**: It provides a lot of built-in functions that make mathematical calculations easy.
- **Efficiency**: With NumPy, you can perform operations on entire lists of data without having to use loops.

{ik} INTERVIEW KICKSTART

NumPy Revisit - Why Use Numpy (Numerical Python) ?

- **Functionality**: It provides a lot of built-in functions that make mathematical calculations easy.

Vs.

{ik} INTERVIEW KICKSTART

NumPy Revisit - Why Use Numpy (Numerical Python) ?

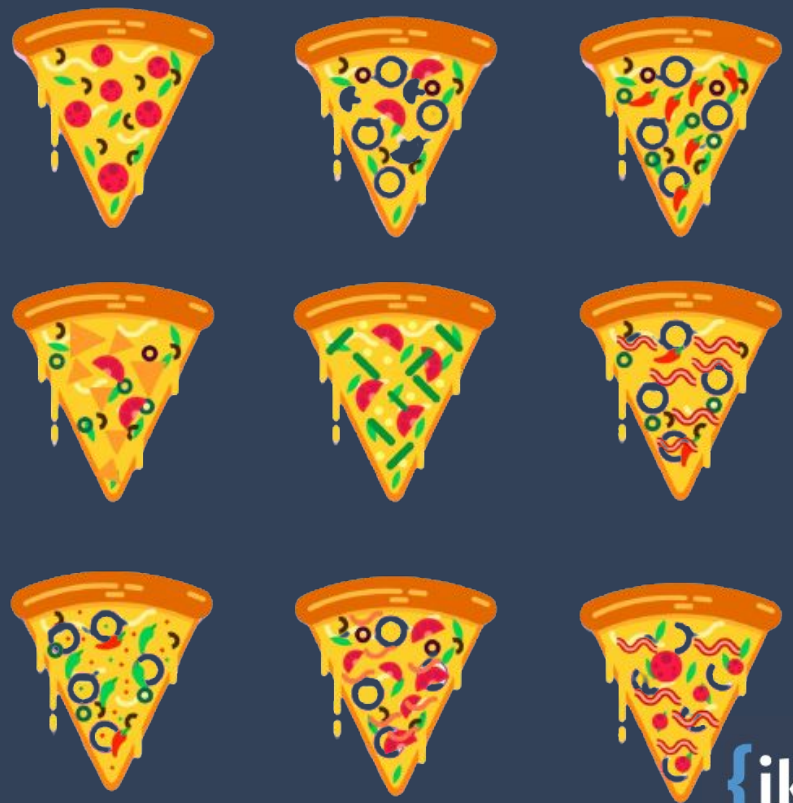
- **Speed**: NumPy is faster than regular Python lists.
- **Functionality**: It provides a lot of built-in functions that make mathematical calculations easy.
- **Efficiency**: With NumPy, you can perform operations on entire lists of data without having to use loops.

{ik} INTERVIEW KICKSTART

NumPy Revisit - Why Use Numpy (Numerical Python) ?

- **Efficiency**: With NumPy, you can perform operations on entire datasets without having to use loops.

Vs.



{ik} INTERVIEW KICKSTART

What we've seen so far in the Foundational Content?

In the NumPy foundational content we've seen -

- NumPy Array
- Reshaping the Array in NumPy
- Indexing & Slicing NumPy Array
- Basic Array Operations
- Broadcasting
- Filtering

Reference Notebook: Introduction to NumPy

Let’s recall a few important topics

In the NumPy foundational content we’ve seen -

- NumPy Array
- Reshaping the Array in NumPy
- Indexing & Slicing NumPy Array
- Basic Array Operations
- Broadcasting
- Filtering

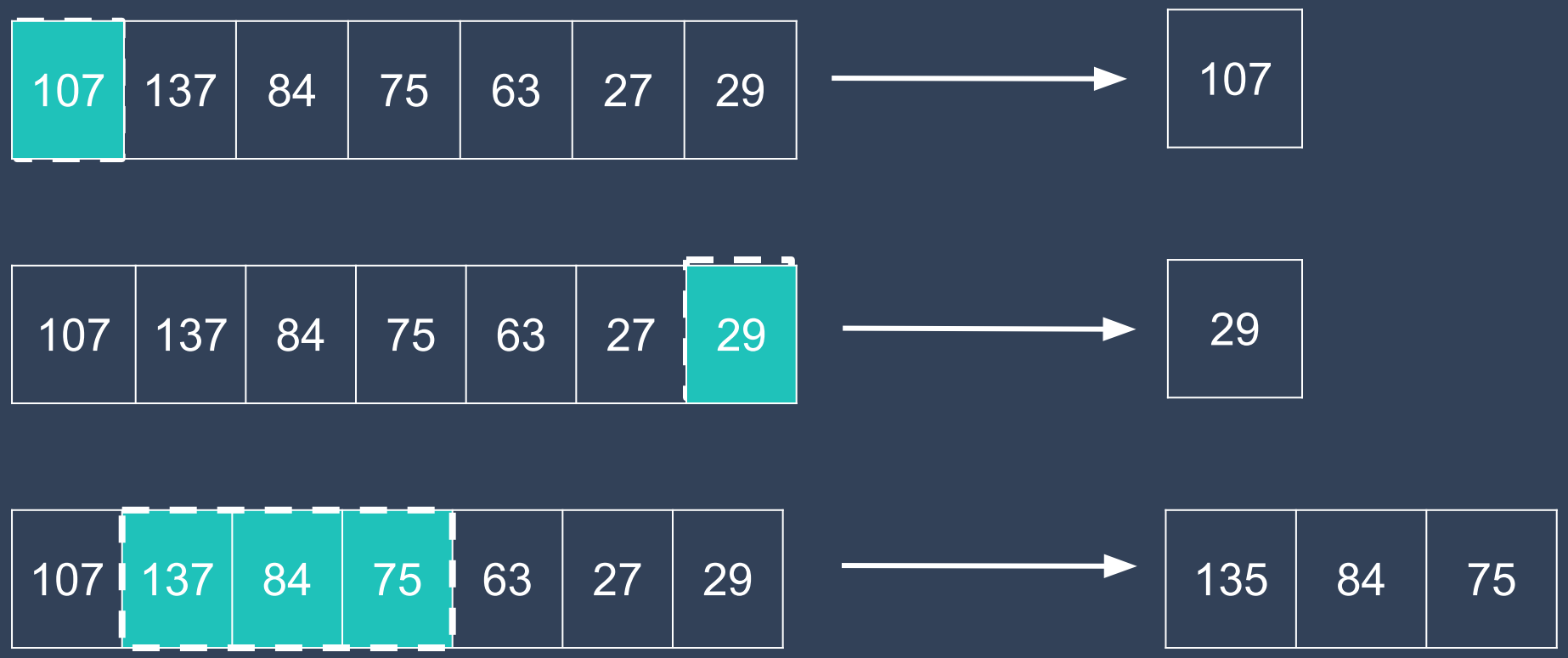
Notebook: Introduction to NumPy

Let’s recall - Indexing and Slicing in Numpy

- **Indexing:** retrieves a single element.
- **Slicing:** retrieves a range of elements.
- Indexing and slicing enable us to extract the relevant features or samples for analysis or model training.

{ik} INTERVIEW KICKSTART

Let's Recall - Indexing and Slicing in Numpy — 1-d array



{ik} INTERVIEW KICKSTART

Let's Recall - Indexing and Slicing in Numpy — 1-d array

Which elements > 30 & < 100?

{ik} INTERVIEW KICKSTART

Let's Recall - Indexing and Slicing in Numpy — 2-d array

26	1	4
0	0	6
0	2	8
0	3	2

→ 26

26	1	4
0	0	6
0	2	8
0	3	2

→ 2

{ik} INTERVIEW KICKSTART

Let's Recall - Indexing and Slicing in Numpy — 2-d array

26	1	4
0	0	6
0	2	8
0	3	2

→

0	2	8

26	1	4
0	0	6
0	2	8
0	3	2

→

1
0
2
3

{ik} INTERVIEW KICKSTART

Let's Recall - Indexing and Slicing in Numpy — 2-d array

26	1	4
0	0	6
0	2	8
0	3	2

→

2	8
3	2

26	1	4
0	0	6
0	2	8
0	3	2

Which elements are odd numbers?

→

1	3

{ik} INTERVIEW KICKSTART

Let’s Recall - Broadcasting in Numpy

- Broadcasting is a powerful feature in NumPy. It describes how NumPy treats arrays with different shapes during arithmetic operations.
- Subject to certain constraints, the smaller array is “broadcast” across the larger array so that they have compatible shapes.
- This broadcasting technique simplifies array operations and makes them more efficient by avoiding the need for explicit loops.

{ik} INTERVIEW KICKSTART

Broadcasting in Numpy - example 1

Coding Challenge: Write a Python function “add_ten”

```
def add_ten(input_arr):
```

26	1	4
0	0	6
0	2	8
0	3	2

→

36	11	14
10	10	16
10	12	18
10	13	12

{ik} INTERVIEW KICKSTART

Broadcasting in Numpy - example 1

Coding Challenge: Write a Python function “add_ten”

26	1	4
0	0	6
0	2	8
0	3	2

+ 10 =

26	1	4
0	0	6
0	2	8
0	3	2

+

10	10	10
10	10	10
10	10	10
10	10	10

=

36	11	14
10	10	16
10	12	18
10	13	12

{ik} INTERVIEW KICKSTART

Broadcasting in Numpy - example 2

26	1	4
0	0	6
0	2	8
0	3	2

$\times$

1	10	100

$=$?

{ik} INTERVIEW KICKSTART

Broadcasting in Numpy - example 2

26	1	4
0	0	6
0	2	8
0	3	2

$\times$

1	10	100

$=$?

26	1	4
0	0	6
0	2	8
0	3	2

$\times$

1	10	100
1	10	100
1	10	100
1	10	100

$=$

26	10	400
0	0	600
0	20	800
0	30	200

{ik} INTERVIEW KICKSTART

Broadcasting in Numpy - example 3

26	1	4
0	0	6
0	2	8
0	3	2

÷

1
3
2
2

= ?

{ik} INTERVIEW KICKSTART

Broadcasting in Numpy - example 3

26	1	4
0	0	6
0	2	8
0	3	2

÷

1
3
2
2

= ?

26	1	4
0	0	6
0	2	8
0	3	2

÷

1	1	1
3	3	3
2	2	2
2	2	2

=

26	1	4
0	0	2
0	1	4
0	1.5	1

{ik} INTERVIEW KICKSTART

Broadcasting in Numpy - example 4

26	1	4
0	0	6
0	2	8
0	3	2

+

1	2	3
4	5	6
7	8	9
10	11	12

= ?

{ik} INTERVIEW KICKSTART

Broadcasting in Numpy - example 4

26	1	4
0	0	6
0	2	8
0	3	2

+

1	2	3
4	5	6
7	8	9
10	11	12

= ?

26	1	4
0	0	6
0	2	8
0	3	2

+

1	2	3
4	5	6
7	8	9
10	11	12

=

27	3	7
4	5	12
7	10	17
10	14	14

{ik} INTERVIEW KICKSTART

Broadcasting in Numpy - General rules

- When operating on two arrays, NumPy compares their shapes *element-wise*. It starts with the trailing dimensions and works its way forward.

- Two arrays A, B are compatible when:
  - one of them is the singular value
  - one is R * C and other is 1 * C
  - one is R * C and other is R * 1
  - they have same shape

{ik} INTERVIEW KICKSTART

QUIZ
TIME!

NumPy Quiz

What distinguishes NumPy arrays from Python lists?

A NumPy arrays do not support operations like element-wise addition or multiplication, which Python lists do.

B Python lists are faster for numerical calculations than NumPy arrays.

C NumPy arrays can only store elements of the same data type, while Python lists can hold elements of different types.

D Python list is designed for efficiency on large arrays of data

{ik} INTERVIEW KICKSTART

NumPy Quiz - Solution

What distinguishes NumPy arrays from Python lists?

A NumPy arrays do not support operations like element-wise addition or multiplication, which Python lists do.

B Python lists are faster for numerical calculations than NumPy arrays.

C NumPy arrays can only store elements of the same data type, while Python lists can hold elements of different types.

D Python list is designed for efficiency on large arrays of data

{ik} INTERVIEW KICKSTART

NumPy Quiz

What are the reasons behind NumPy's high performance for numerical computations?

A NumPy operations are implemented in Python itself, leveraging Python's dynamic typing and interpretative nature for flexibility.

B NumPy utilizes dynamic typing and runs purely in the Python Virtual Machine (PVM) for better integration with Python code.

C NumPy is built on the Python Data Analysis Library (Pandas), which provides fast, flexible data structures.

D NumPy uses a homogeneous array structure and is optimized with C and Fortran, enabling efficient memory usage and vectorized operations.

{ik} INTERVIEW KICKSTART

NumPy Quiz - Solution

What are the reasons behind NumPy's high performance for numerical computations?

A NumPy operations are implemented in Python itself, leveraging Python's dynamic typing and interpretative nature for flexibility.

B NumPy utilizes dynamic typing and runs purely in the Python Virtual Machine (PVM) for better integration with Python code.

C NumPy is built on the Python Data Analysis Library (Pandas), which provides fast, flexible data structures.

D NumPy uses a homogeneous array structure and is optimized with C and Fortran, enabling efficient memory usage and vectorized operations.

{ik} INTERVIEW KICKSTART

{ik} INTERVIEW KICKSTART

Today's Agenda

1. NumPy Revisit

Break

2. Pandas Fundamentals

Break

3. Pandas Data Analysis

Break

4. Plotting with Matplotlib

{ik} INTERVIEW KICKSTART

Pandas Fundamental

Pandas Fundamental

- What is Pandas
- Why use Pandas
- Data Structures in Pandas

What is Pandas (comes from Panel Data)

- Pandas is a powerful Python library widely used for *data manipulation* and *analysis*.

- Pandas offers data structures and functions that simplify working with structured data, making it an essential tool in data science and analysis

{ik} INTERVIEW KICKSTART

Why use Pandas?

- Using Pandas offers several benefits for data manipulation and data analysis:
  - Efficient Data Handling
  - Data Alignment
  - Handling Missing Data
  - Data Integration
  - Flexible Data Transformation
  - Integration with Other Libraries

{ik} INTERVIEW KICKSTART

Data Structures in Pandas

- Pandas **Series** is a one-dimensional array with axis labels.
- Pandas **DataFrame** is two-dimensional data structure with labeled rows and columns.

Series A

	apples
0	3
1	2
2	0
3	1

+

Series B

	oranges
0	0
1	3
2	7
3	2

=

DataFrame

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

{ik} INTERVIEW KICKSTART

Series and DataFrame Creation in Pandas

Time to Code!

{ik} INTERVIEW KICKSTART

Filtering a Dataframe - Why do we need filtering?

- When we need to remove redundant or unnecessary data for some tasks
- When we want to filter customers based on certain criteria
- When we need to filter out rows or columns have missing values

{ik} INTERVIEW KICKSTART

Filtering a Data Frame

- Filtering with `**loc**` and `**iloc**` methods
- Filtering by Selecting a Subset of Columns
- Filtering by condition(s)

{ik} INTERVIEW KICKSTART

Filtering a Dataframe - `loc` and `iloc` methods

- **loc** uses row and column labels.
- **iloc** uses row and column indexes.

	column index		0	1	2
row index	label		Name	Age	Education
0	March	2			
1	April	3			
2	May	4			
3	June	5			
4	July	6			

{ik} INTERVIEW KICKSTART

Filtering a Data Frame - Selecting a Subset of Columns

- Select a single column
- Select multiple columns

	state	account_length	international_plan	total_intl_calls	churn
0					
1					
2					

Select multiple columns

Select a column

{ik} INTERVIEW KICKSTART

Filtering a Dataframe - Filtering by condition(s)

- Filtering by a condition
- Filtering by multiple conditions

The operators we can use in filtering by condition(s):

- ==: equal
- !=: not equal
- >: greater than
- >=: greater than or equal to
- <: less than
- <=: less than or equal to
- & : and
- | : or

{ik} INTERVIEW KICKSTART

Handling Missing Values with Pandas

- Missing Value Types and Representation
- Finding the Missing Values
- Dropping Rows and Columns with Missing Values
- Replacing the Missing Values

{ik} INTERVIEW KICKSTART

Handling Missing Values with Pandas

- Missing values are essentially data we don’t have
- NaN : stands for “Not a Number”
  - Python’s **None**
  - **np.nan**

{ik} INTERVIEW KICKSTART

Finding the Missing Values

- isna() function: returns **True** to indicate a missing value
- notna() function: returns **False** for missing values

	state	account_length	churn
0	CA	314	yes
1	CO	?	no
2	?	44	yes

{ik} INTERVIEW KICKSTART

Dropping Rows and Columns with Missing Values

- dropna() function:
  - `axis`: 0 (default value, indicates row), 1
  - `how`: “any” (default value), “all”
  - `thresh`: a row or column that at least X non-missing values will be kept, otherwise will be dropped.

	state	account_length	churn
0	CA	314	?
1	CO	?	?
2	?	44	?

{ik} INTERVIEW KICKSTART

Replacing the Missing Values

- Dropping may not be the best option in many cases
- Replace missing values with:
  - the average value of the column
  - the most frequent value in the column
  - a random number
- fillna() function

	state	account_length	churn
0	CA	314	Yes
1	CO	100	Yes
2	CO	44	no

{ik} INTERVIEW KICKSTART

Sorting in a Data Frame

- Sort Data Frames based on their column features
- sort_values() : sort a DataFrame by one or more of its columns
  - First argument: either a column label or a list of column labels
  - `ascending` : ascending (default value), descending

	Name	age
0	David	33
1	Emily	20
2	Emma	44

	Name	age
0	Emily	20
1	David	33
2	Emma	44

{ik} INTERVIEW KICKSTART

Data Manipulation with Pandas

- drop(): Remove rows or columns from a DataFrame
- replace(): Replace values within a DataFrame
- apply(): Applying a given function along an axis of a DataFrame or Series.
- Add a new column from current data

	user_id	account_length	total_orders	total_amt
0	1232as	31	4	156
1	1323wv	0	2	127
2	134pdf	0	3	135
3	342t6ff	23	4	466

INTERVIEW KICKSTART

QUIZ
TIME!

Pandas Quiz

Which of the following is the correct way to create a DataFrame in pandas from a dictionary?

A df = pd.DataFrame(data=[{'a': 1, 'b': 2}, {'a': 3, 'b': 4}])

B df = pd.DataFrame({'a': [1, 3], 'b': [2, 4]})

C df = pd.DataFrame(data=[1, 2, 3, 4], columns=['a', 'b'])

D df = pd.DataFrame(columns={'a': [1, 3], 'b': [2, 4]})

{ik} INTERVIEW KICKSTART

Pandas Quiz

Which of the following is the correct way to create a DataFrame in pandas from a dictionary?

A df = pd.DataFrame(data=[{'a': 1, 'b': 2}, {'a': 3, 'b': 4}])

B df = pd.DataFrame({'a': [1, 3], 'b': [2, 4]})

C df = pd.DataFrame(data=[1, 2, 3, 4], columns=['a', 'b'])

D df = pd.DataFrame(columns={'a': [1, 3], 'b': [2, 4]})

{ik} INTERVIEW KICKSTART

Pandas Quiz

What will df.iloc[0:3, 1:3] return?

A The first 3 rows and all columns

B The first 3 columns and all rows

C A subset with the first 3 rows and columns 1 and 2

D A subset with the first 3 rows and columns 1 to 3

{ik} INTERVIEW KICKSTART

Pandas Quiz

What will df.iloc[0:3, 1:3] return?

- **A** The first 3 rows and all columns
- **B** The first 3 columns and all rows
- **C** A subset with the first 3 rows and columns 1 and 2
- **D** A subset with the first 3 rows and columns 1 to 3

{ik} INTERVIEW KICKSTART

{ik} INTERVIEW KICKSTART

Today's Agenda

1. NumPy Revisit

Break

2. Pandas Fundamentals

Break

3. Pandas Data Analysis

Break

4. Plotting with Matplotlib

{ik} INTERVIEW KICKSTART

Pandas: Data Analysis

Data Analysis with Pandas

- Data Analysis: The process of inferring insights, discovering useful information, and drawing results from the data
  - groupby(): groups rows of a DataFrame based on one or more columns
  - agg(): aggregate data within groups created by `groupby()`
  - reset_index(): reset the index of a DataFrame
  - rename(): change the labels (names) of rows or columns in a DataFrame
  - idxmax(): Return index of first occurrence of maximum over requested axis

	total_order	age
0	20	33
1	40	20
2	100	44

Decision-Making process

Explore the data before creating ML model

{ik} INTERVIEW KICKSTART

Data Analysis with Pandas - groupby(), agg()

- The `groupby` function creates the groups, but they don't provide any information unless we do some aggregations.

DataFrame

product_group	product_code	price
A	1001	9
A	1002	14
B	1101	21
A	1003	12
B	1104	19
C	1201	7
B	1105	25

groupby("product_group")

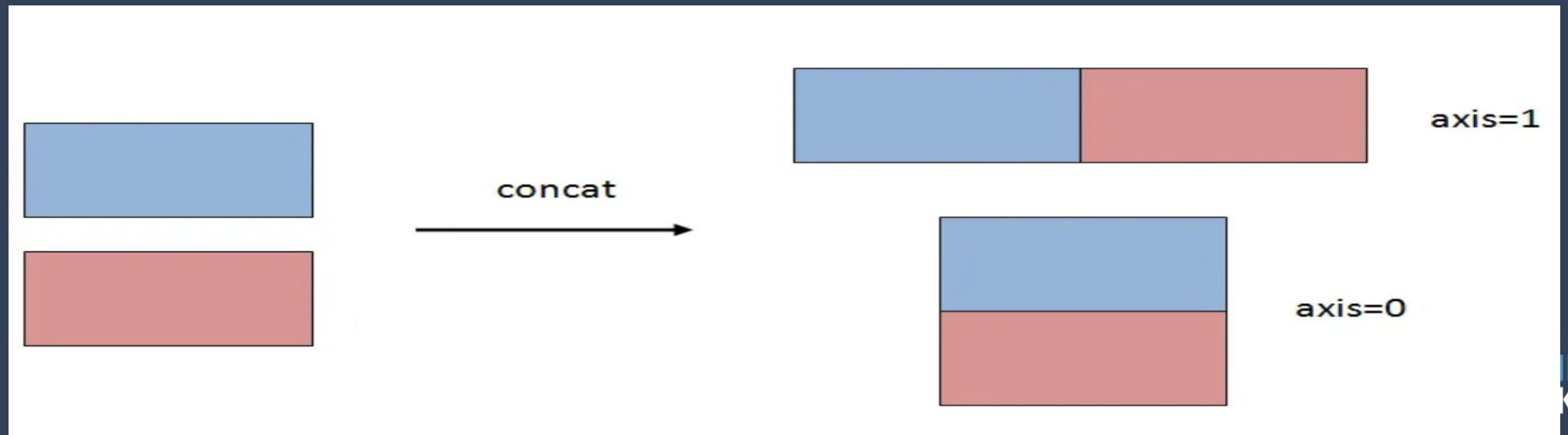
product_group	product_code	price
A	1001	9
A	1002	14
A	1003	12

product_group	product_code	price
B	1101	21
B	1104	19
B	1105	25

product_group	product_code	price
C	1201	7

Combining DataFrames with Pandas - The concat function

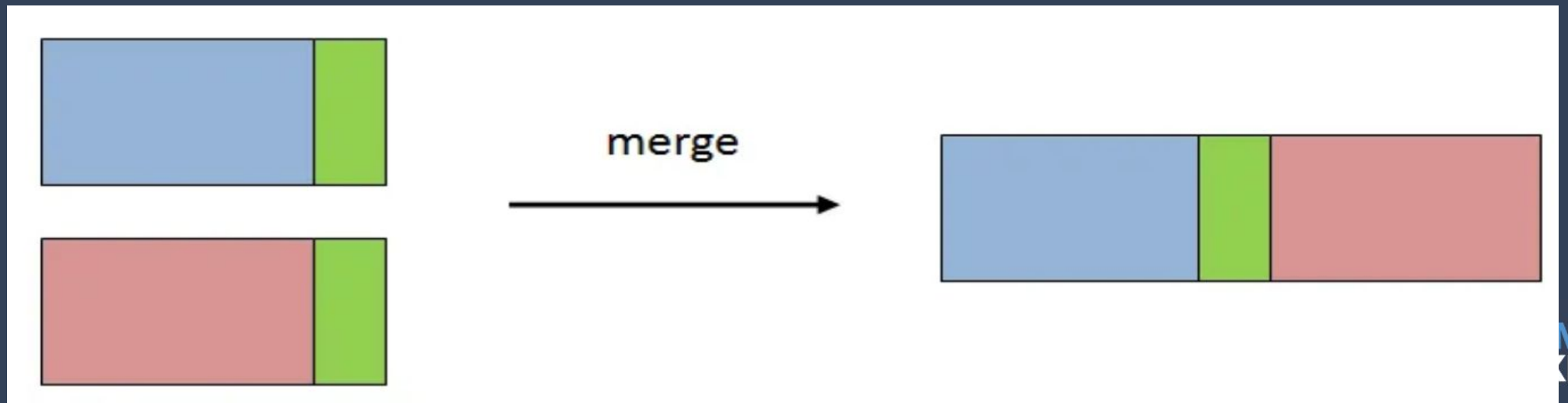
- Concatenation is the process of combining DataFrames along a particular axis (either vertically or horizontally).
- When you concatenate DataFrames, you're essentially stacking them on top of each other (vertically) or side by side (horizontally).
- The resulting DataFrame maintains the original column names and indexes of the individual DataFrames.



INTERVIEW KICKSTART

Combining DataFrames with Pandas - The merge function

- Merging is the process of combining DataFrames based on common columns or indexes.
- It is often used to combine DataFrames with related information, similar to how SQL JOIN operation works.
- Merging allows us to specify which columns to use as keys for merging and how to handle overlapping column names.



NTERVIEW
KICKSTART

QUIZ
TIME!

Pandas Quiz

What does the groupby method return when applied to a DataFrame?

A A DataFrame

B A DataFrameGroupBy object

C A Series

D A DataFrameIterator

{ik} INTERVIEW KICKSTART

Pandas Quiz

What does the groupby method return when applied to a DataFrame?

A A DataFrame

B A DataFrameGroupBy object

C A Series

D A DataFrameIterator

{ik} INTERVIEW KICKSTART

Pandas Quiz

What will the following code do? pd.concat([df1, df2], axis=1)

A Concatenate df1 and df2 along rows

B Concatenate df1 and df2 along column

C Merge df1 and df2 on a common key

D Append df2 to the end of d

{ik} INTERVIEW KICKSTART

Pandas Quiz

What will the following code do? pd.concat([df1, df2], axis=1)

A Concatenate df1 and df2 along rows

B Concatenate df1 and df2 along column

C Merge df1 and df2 on a common key

D Append df2 to the end of d

{ik} INTERVIEW KICKSTART

Today's Agenda

1. NumPy Revisit

Break

2. Pandas Fundamentals

Break

3. Pandas Data Analysis

Break

4. Plotting with Matplotlib

{ik} INTERVIEW KICKSTART

Plotting with Matplotlib

What is Matplotlib

- Matplotlib is a popular Python library for data visualization and plotting.
- It provides a wide range of tools for creating static, animated, and interactive plots.

{ik} INTERVIEW KICKSTART

Why to Use Matplotlib ?

- Matplotlib is easy to use and integrates well with other Python libraries like NumPy and Pandas.
- It supports a variety of plot types, making it suitable for various data visualization tasks.
- Matplotlib offers complete control over plot customization, allowing you to create publication-quality graphics.
- matplotlib.pyplot

{ik} INTERVIEW KICKSTART

Basic Plot Types

- Line Chart
- Scatter Chart
- Bubble Chart
- Histogram
- Bar Chart
- Pie Chart
- Heatmap

{ik} INTERVIEW KICKSTART

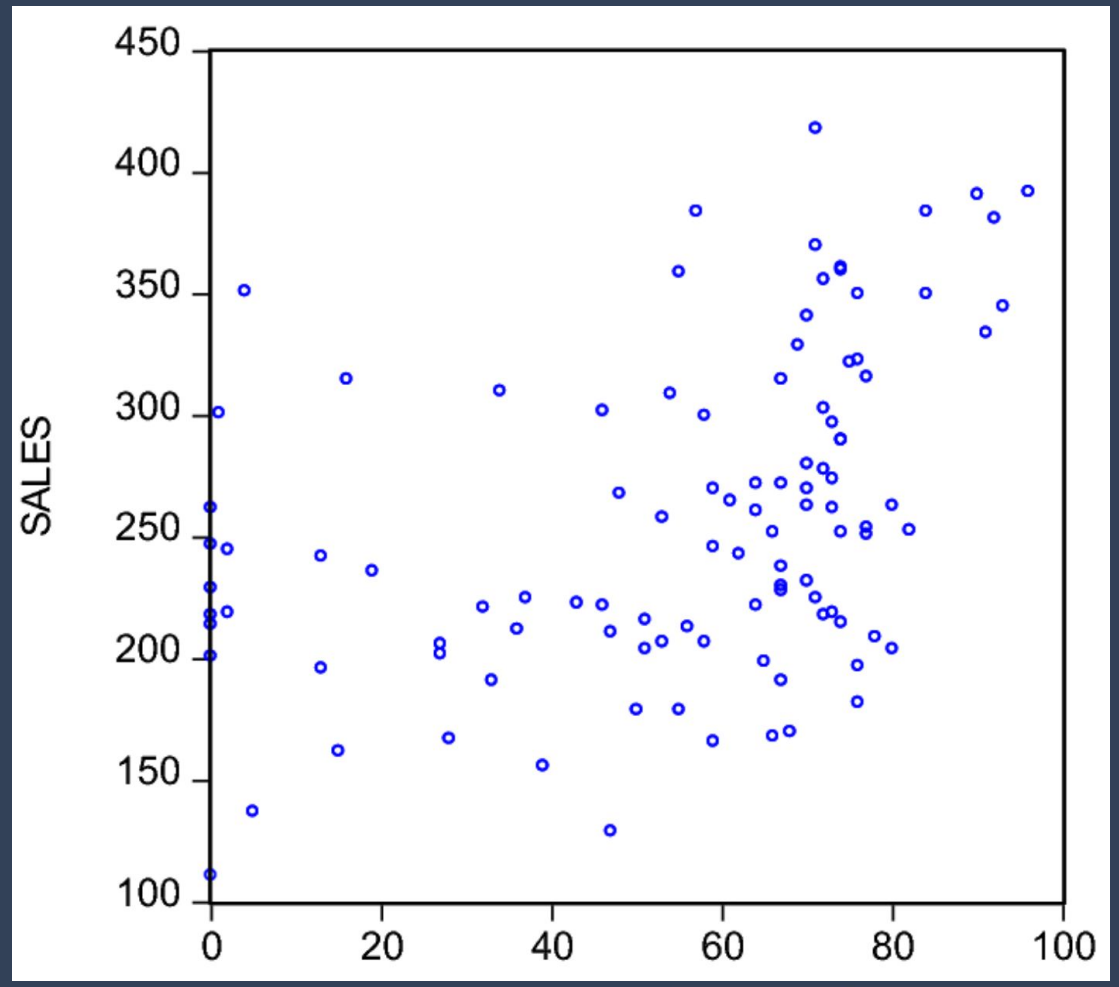
Line Chart

- Use Case: Tracking stock prices over time.
- X-axis: Time(any ordered series).
- Y-axis: Stock Price (A continuous variable showing progression or change over the X-axis.).
- Best Scenario: To display trends or changes over time.

{ik} INTERVIEW KICKSTART

Scatter Chart

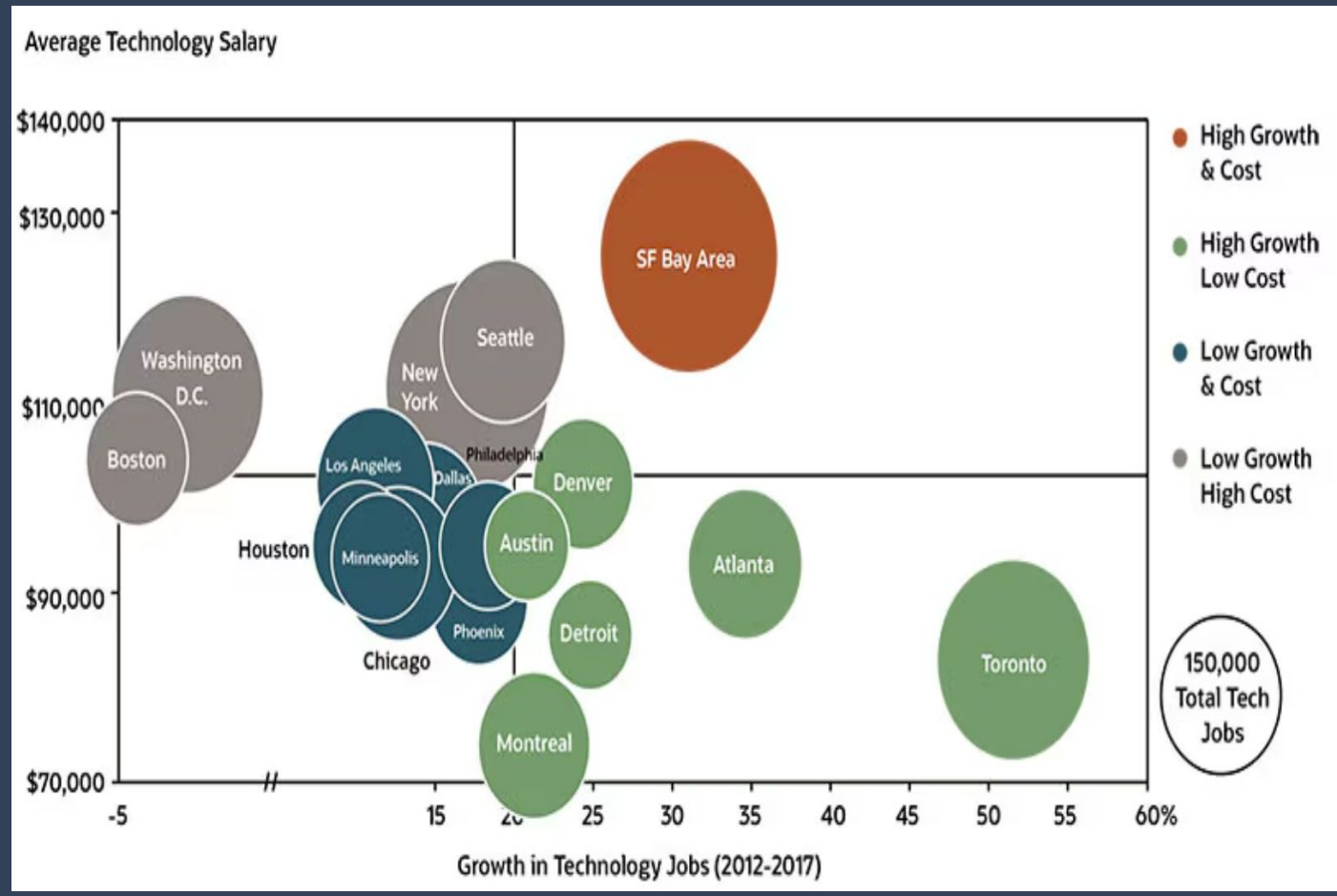
- Use Case: UberEats analyzing the relationship between advertising spending on Google ads and the revenue generated.
- X-axis: Advertising Cost
- Y-axis: Sales
- Best Scenario: To explore potential relationships or correlations between two continuous variables.



{ik} INTERVIEW KICKSTART

Bubble Chart

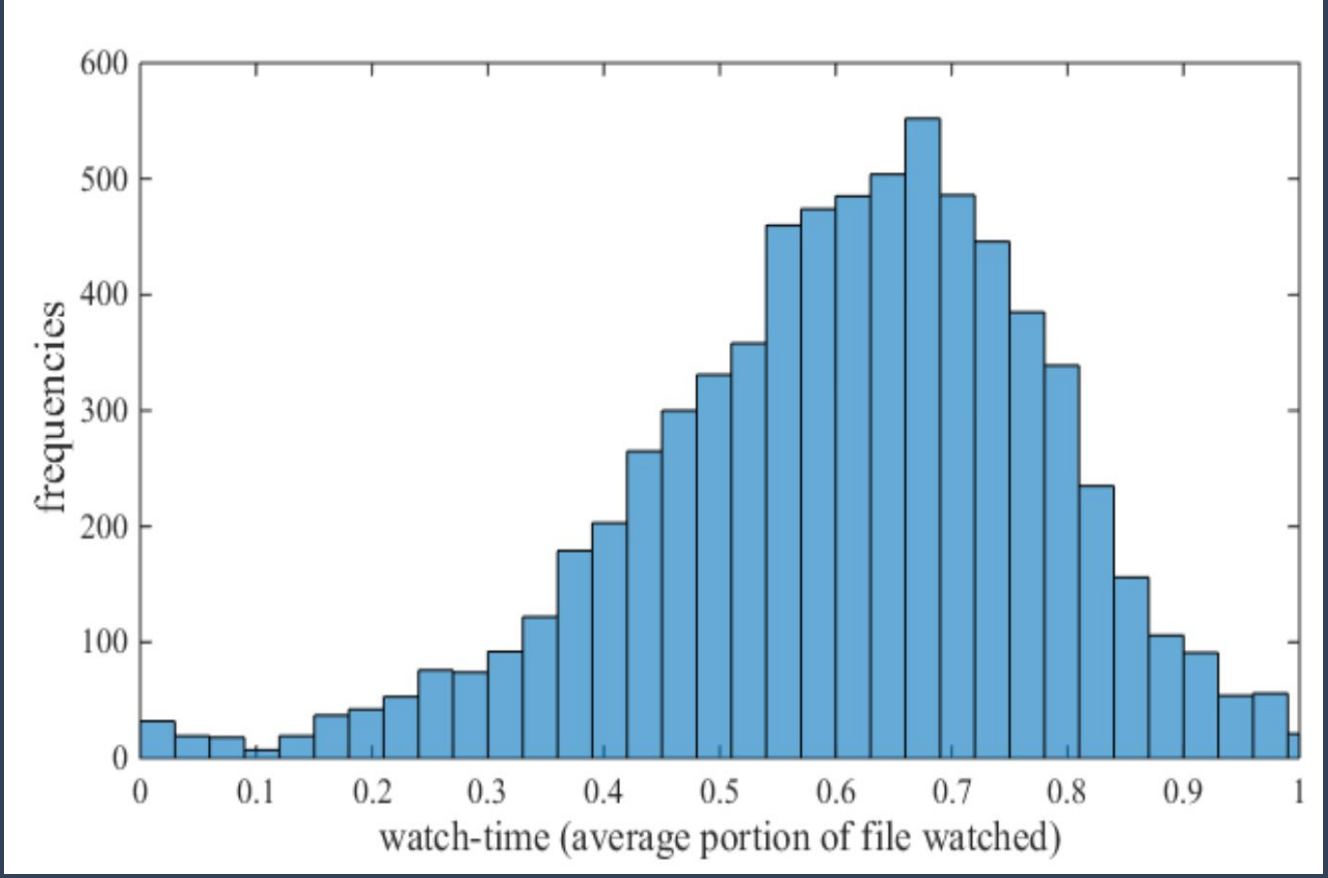
- Use Case: To find an affordable city for a tech job.
- X-axis: Growth in Tech jobs
- Y-axis: Avg Tech Salary
- Bubble size: # of job openings
- Best Scenario: To find "attractive" clusters considering multiple factors.



{ik} INTERVIEW KICKSTART

Histogram

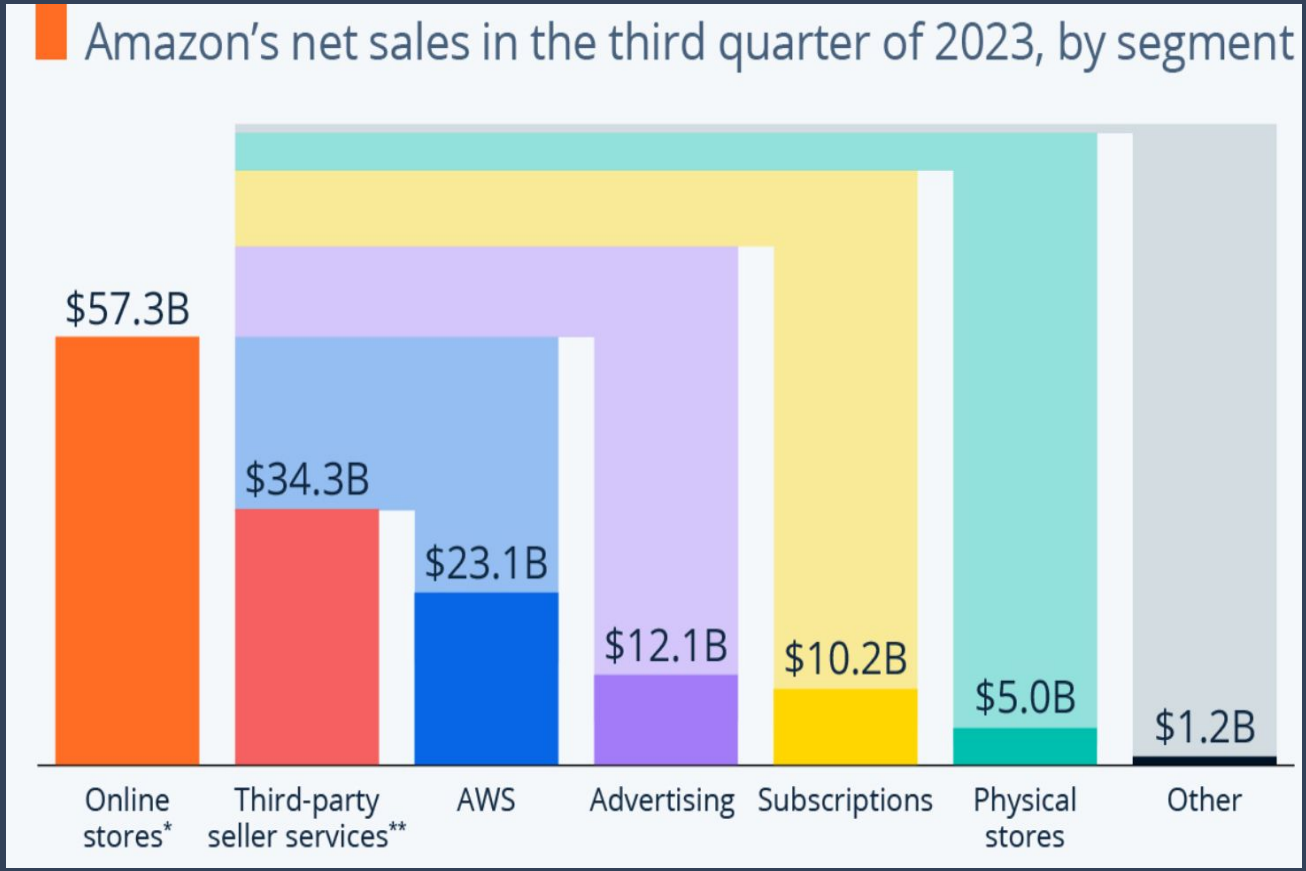
- Use Case: Understand the distribution of users watch time on Youtube
- X-axis: Watch time on Youtube(continuous variable)
- Y-axis: Frequency(# of users)
- Best Scenario: To understand the distribution of a dataset.



{ik} INTERVIEW KICKSTART

Bar Chart

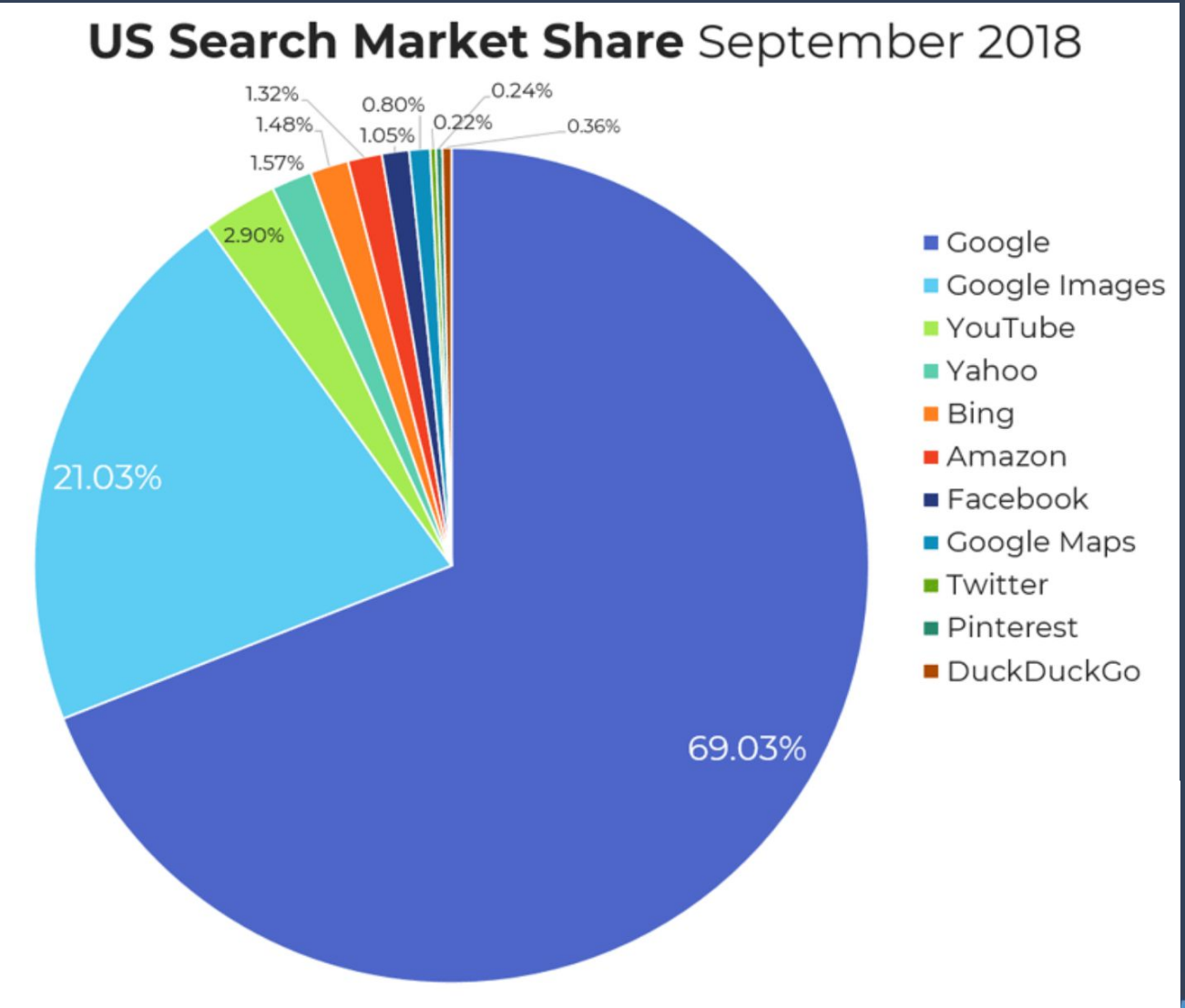
- Use Case: Understand Amazon net sales by segment
- X-axis: segments
- Y-axis: Net sales amount($)
- Best Scenario: To compare quantities across categories.



{ik} INTERVIEW KICKSTART

Pie Chart

- Use Case: Understand the US Search Market Share
- Best Scenario: To show parts of a whole in percentage



{ik} INTERVIEW KICKSTART

Heatmap

- Use Case: Analyzing peak usage times on Instagram throughout the week
- Two dimensions: hours of the day vs. days of the week
- Color bar: Indicating user activity levels
- Best Scenario: Understand how performance changes over the hours of the day and the days of the week

	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
0	6.00	7.35	11.79	9.82	13.93	7.60	9.10
1	3.73	7.29	7.88	6.62	8.73	5.58	5.63
2	3.44	7.31	5.29	4.23	3.98	3.70	3.06
3	3.67	7.02	3.69	3.12	1.27	2.77	1.94
4	3.81	5.50	1.92	2.65	1.42	3.55	2.29
5	4.54	3.67	1.29	2.48	2.63	5.40	3.58
6	5.52	3.46	3.46	3.67	3.68	6.53	4.33
7	5.75	6.15	7.33	6.23	5.70	6.58	4.85
8	7.94	10.46	10.79	10.40	8.92	6.10	6.25
9	13.58	14.08	14.29	14.13	11.97	7.10	7.38
10	18.77	16.27	18.83	15.70	15.02	10.10	8.23
11	21.00	17.52	24.77	15.25	18.60	12.68	10.10
12	21.13	18.38	31.77	15.27	22.12	13.88	12.60
13	22.54	20.38	36.85	18.55	23.77	14.35	12.81
14	27.02	24.02	37.40	24.62	23.70	14.28	10.15
15	30.52	26.58	33.00	30.13	23.13	13.20	8.15
16	29.42	25.77	25.56	29.78	20.40	11.62	9.46
17	23.67	22.81	19.04	23.28	15.42	11.65	12.63
18	17.88	19.79	15.23	17.08	11.15	13.13	13.92
19	16.38	16.69	14.06	16.17	9.93	13.78	12.04
20	16.04	13.21	14.75	18.90	10.08	13.25	9.58
21	13.77	11.92	14.96	20.88	9.62	12.92	9.69
22	10.48	13.98	14.38	20.17	8.92	12.66	10.98
23	8.00	14.69	13.27	17.85	8.48	11.43	9.58

QUIZ
TIME!

Matplotlib Quiz

How can you display the plot in a Matplotlib script?

A plt.showplot()

B plt.render()

C plt.display()

D plt.show()

{ik} INTERVIEW KICKSTART

Matplotlib Quiz

How can you display the plot in a Matplotlib script?

A plt.showplot()

B plt.render()

C plt.display()

D plt.show()

{ik} INTERVIEW KICKSTART

Matplotlib Quiz

How can you customize the line style in a plot?

A plt.plot(x, y, linestyle='dash')

B plt.plot(x, y, linestyle='--')

C plt.plot(x, y, style='dash')

D plt.plot(x, y, line='dash')

{ik} INTERVIEW KICKSTART

Matplotlib Quiz

How can you customize the line style in a plot?

A plt.plot(x, y, linestyle='dash')

B plt.plot(x, y, linestyle='--')

C plt.plot(x, y, style='dash')

D plt.plot(x, y, line='dash')

{ik} INTERVIEW KICKSTART

Learning by Doing!

Userful Resources:

- Numpy documentation
- Pandas documentation
- Matplotlib documentation
- Leetcode
- Medium
- Engineering Blogs

{ik} INTERVIEW KICKSTART

{ik} INTERVIEW KICKSTART